

Automation of a Current-Mode Bandgap Reference Using gm/Id-Based Synthesis, Simulation-Driven Optimization, and Noise-Aware Optimization

Jaiveer Kiran S K

Department of Electrical Engineering
Indian Institute of Technology Madras
Email: ee22b042@smail.iitm.ac.in

Abstract—This work presents a fully automated design flow for a CMOS current-mode bandgap reference based on a combination of analytical pre-sizing, gm/Id-driven device synthesis, and simulation-in-the-loop optimization. The flow integrates Python scripts with Cadence Spectre to (i) compute initial resistor ratios and device dimensions from bandgap equations and diode characterization, (ii) automatically synthesize transistor fingers from gm/Id look-up tables, (iii) iteratively optimize the bandgap’s temperature characteristic using a three-stage algorithm that enforces smoothness, nominal reference voltage, and a constrained turning-point window, and (iv) refine the design for total output noise and 1/f noise corner using per-device noise contribution analysis. For a 1.2 V supply and a 0.5 V target reference, the automated flow converges to a solution that meets specifications on temperature coefficient, output voltage variation, power consumption, and noise corner with minimal manual intervention.

Index Terms—Bandgap reference, CMOS analog design, gm/Id methodology, circuit automation, noise optimization, Python scripting, Cadence Spectre.

I. INTRODUCTION

Bandgap reference (BGR) circuits are essential building blocks in mixed-signal systems, providing a temperature-stable voltage reference for voltage regulators, data converters, and sensor interfaces. Achieving low temperature coefficient, tight reference accuracy, and competitive noise performance traditionally relies on manual iteration across device sizing, resistor ratios, and bias currents, which is time-consuming and error-prone, especially in scaled CMOS processes.

Recent work has explored automated or algorithmic support for analog design, including global optimization and machine-learning-assisted flows for analog blocks and bandgap references, as well as metaheuristic sizing methods such as particle-swarm or firefly algorithms. These approaches typically treat the circuit as a black box and optimize a parameter set directly against a cost function.

In contrast, this work encodes bandgap and gm/Id design knowledge into a staged automation flow that remains closely aligned with manual design practice. The main contributions are:

- An analytical pre-sizing stage that derives resistor values, op-amp gain requirements, and device lengths from bandgap equations and a single diode DC sweep.
- A gm/Id-based synthesis stage that automatically computes transistor fingers for the op-amp, bias, and mirror devices from Spectre-generated gm/Id look-up tables.
- A three-stage simulation-driven bandgap optimizer that enforces smoothness of $V_{out}(T)$, centers the reference voltage, and constrains the turning-point temperature to a specified range.
- A noise and 1/f-corner optimization stage that parses Spectre noise summaries, identifies dominant contributors, and selectively increases device areas to meet noise and corner targets while preserving the DC behavior.

II. DESIGN FLOW OVERVIEW

The proposed automation flow is implemented as a Python front-end that orchestrates three main steps: (i) analytical pre-sizing, (ii) gm/Id-based device synthesis, and (iii) simulation-in-the-loop optimization of DC and noise behavior. The user interacts only with a specification file (`spec.py`) that defines the maximum power, nominal supply, target reference voltage, allowed voltage error, and optional noise constraints.

Internally, the framework executes the following sequence:

- 1) Run a diode temperature sweep (`diode.scs`) to extract $V_{BE}(T)$, compute the CTAT slope, and derive initial resistor values R_1 , R_2 , and R_3 .
- 2) Compute current budgets, op-amp gain and headroom constraints, mirror output resistance, and required Early voltages using closed-form expressions.
- 3) Run NMOS/PMOS characterization netlists (`nmoschar.scs`, `pmoschar.scs`) to build gm/Id look-up tables and map target gm/Id to per-finger currents.
- 4) Synthesize finger counts and lengths for op-amp, mirror, and bias devices, and generate a complete parameter set for the bandgap netlist.
- 5) Invoke a three-stage optimizer that repeatedly edits the bandgap netlist (`band_gap.scs`), runs Spectre tem-

perature sweeps, and updates mirror fingers and resistor values until the $V_{out}(T)$ curve is smooth, centered, and has a turning point in the desired range.

- 6) Optionally, run noise analysis and a noise-aware refinement loop that parses per-device noise contributions and properly do the required changes in dimensions of dominant devices based on the type of noise (flicker/thermal) until integrated noise, 1/f Corner and spot noise requirements are met.

All Spectre runs are controlled from Python via `subprocess` calls, and PSF ASCII outputs are parsed directly to avoid any manual interaction with the CAD environment.

III. CIRCUIT IMPLEMENTATION

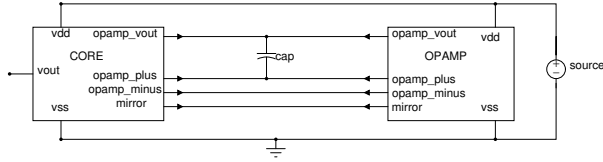


Fig. 1. Top-level schematic.

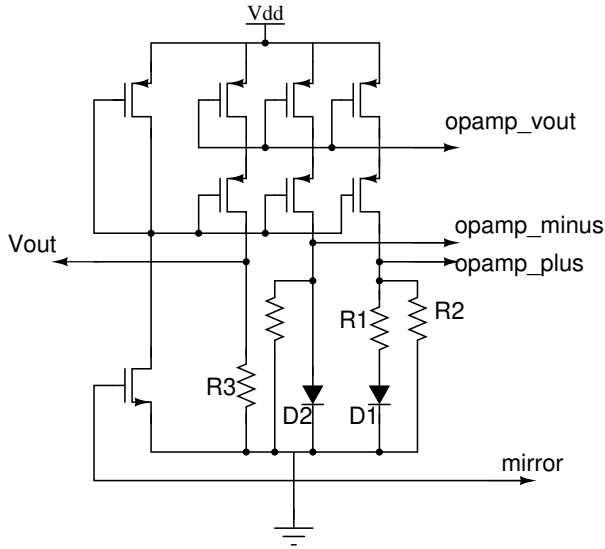


Fig. 2. CORE.

IV. PRE-SIZING METHODOLOGY

A. Bandgap equations and resistor computation

The flow begins with an analytical computation of the CTAT slope from a diode DC simulation. A dedicated netlist (`diode.scs`) is run with a branch current I_{branch} , and the script extracts the base-emitter voltage $V_{BE}(T)$ at various temperatures from the PSF file. From the extracted lookup table $\{T : V_{BE}\}$, the CTAT slope is computed as

$$CTAT_slope = \frac{V_{BE}(125^\circ C) - V_{BE}(-40^\circ C)}{165^\circ C} \quad (1)$$

and the reference point $V_{BE}(30^\circ C)$ is recorded.

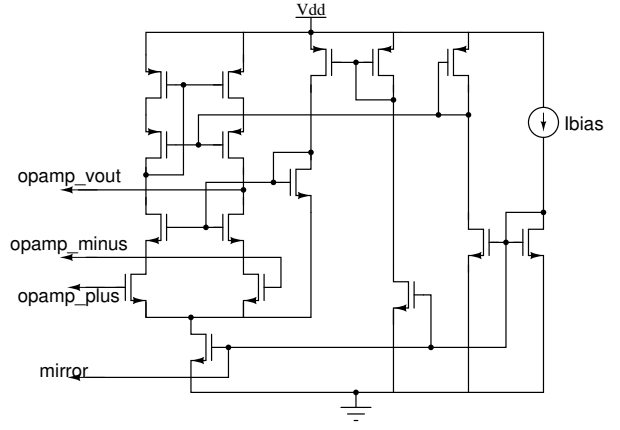


Fig. 3. OPAMP.

Using the standard bandgap relation

$$V_{ref} = V_{BE}(30^\circ C) + \alpha \Delta V_{BE}, \quad (2)$$

with

$$\Delta V_{BE} \approx \eta \frac{kT}{q} \ln(n), \quad (3)$$

the resistor ratio is obtained as

$$\frac{R_2}{R_1} = -\frac{CTAT_slope}{\eta \frac{k}{q} \ln(n)}. \quad (4)$$

Given the design current I and target reference voltage V_{ref} , R_1 , R_2 , and R_3 follow from closed-form expressions derived in the sizing script.

B. Current budget and op-amp requirements

A maximum power constraint from the specification file (`spec.Max_power`, `spec.vdd`) is translated into a total design current

$$I_{max} = \frac{P_{max}}{V_{DD}}, \quad I_{design} = 0.9 I_{max}. \quad (5)$$

The script splits I_{design} into an op-amp branch current, a bias current, and a bandgap core current. A fixed fraction of the allowed reference error budget is assigned to current mirror inaccuracy ($\approx 10\%$) and the remainder to finite op-amp gain ($\approx 90\%$), which directly sets targets on mirror output resistance and op-amp loop gain.

Using the mirror error budget, the allowed variation in branch current is

$$\Delta I = \frac{V_{ref} \cdot (\Delta V_{ref,mirror}[\%]/100)}{R_3}, \quad (6)$$

and the required mirror output resistance is

$$R_{out} = \frac{V_{BE}(-40^\circ C) - V_{ref}}{\Delta I}. \quad (7)$$

The worst-case low supply voltage is

$$V_{DD,min} = V_{DD} \left(1 - \frac{\Delta V_{DD}[\%]}{100}\right), \quad (8)$$

and the minimum mirror saturation voltage is chosen as

$$V_{\text{DSAT,mirror}} = \frac{V_{\text{DD,min}} - V_{\text{BE}}(-40^\circ\text{C})}{2}, \quad (9)$$

leading to a target mirror gm/Id

$$\left(\frac{g_m}{I_D}\right)_{\text{mirror}} = \frac{2}{V_{\text{DSAT,mirror}}}. \quad (10)$$

For the op-amp, the allotted error on V_{ref} is back-propagated to an admissible input differential error, yielding a required closed-loop gain A and corresponding

$$A_{\text{dB}} = 20 \log_{10} A. \quad (11)$$

A target op-amp saturation voltage $V_{\text{DSAT,opamp}}$ sets

$$\left(\frac{g_m}{I_D}\right)_{\text{opamp}} = \frac{2}{V_{\text{DSAT,opamp}}}, \quad (12)$$

and the combination of R_{out} , g_m/I_D , and gain determines a required Early voltage for each device class. Pre-tabulated $V_A(L)$ curves are then interpolated to obtain channel lengths for mirrors and op-amp devices.

V. GM/ID-BASED SYNTHESIS AND CORE OPTIMIZATION

A. gm/Id characterization and finger computation

The automation performs gm/Id characterization for NMOS and PMOS transistors by running Spectre on dedicated netlists (`nmoschar.scs`, `pmoschar.scs`) and parsing the resulting PSF ASCII files into Id - gm/Id lookup tables. For a given target gm/Id , the per-finger current is obtained by nearest-neighbor search in the table, and the required number of fingers is

$$N_{\text{fingers}} = \text{round}\left(\frac{I_{\text{branch}}}{I_{\text{per-finger}}}\right). \quad (13)$$

This gm/Id -based synthesis determines finger counts for op-amp input and bias devices, bandgap current mirror devices (including bias legs), and op-amp tail devices. A function `run_iteration()` assembles these counts with the analytically derived lengths and resistor values into a complete parameter set for the bandgap netlist.

B. Three-stage simulation-driven bandgap optimization

The core bandgap optimization is encapsulated in `optimize_bandgap()`, which:

- edits the bandgap netlist with the current parameters,
- runs Spectre for a temperature sweep, and
- evaluates $V_{\text{out}}(T)$ from the PSF DC file.

1) *Stage 1: Smoothness enforcement*: A smoothness cost is defined as

$$J_{\text{smooth}} = \sum_k |\Delta^2 V_{\text{out}}(T_k)|, \quad (14)$$

i.e., the sum of absolute second differences of $V_{\text{out}}(T)$ over the simulated temperature points. The NMOS mirror bias finger count $f_{n_m_b}$ is incremented until J_{smooth} falls below a tolerance, enforcing a single well-behaved CTAT/PTAT combination.

2) *Stage 2: Reference voltage centering*: The output voltage at 30°C , $V_{\text{out}}(30^\circ\text{C})$, is read directly or interpolated from the sweep. The PMOS mirror finger count f_{p_m} is iteratively adjusted until

$$V_{\text{ref,min}} \leq V_{\text{out}}(30^\circ\text{C}) \leq V_{\text{ref,max}}, \quad (15)$$

where the bounds are defined by a user-specified error margin around the target V_{ref} . Configurations that violate the smoothness constraint are rejected.

3) *Stage 3: Turning-point window and resistor refinement*: The turning-point temperature T_p is estimated from $V_{\text{out}}(T)$ by detecting sign changes in the discrete slope and interpolating between adjacent temperature samples. A turning-point cost

$$J_{\text{tp}}(T_p) = \begin{cases} 1, & T_p = \text{None} \\ \frac{T_{\text{min}} - T_p}{T_{\text{max}} - T_{\text{min}}}, & T_p < T_{\text{min}} \\ \frac{T_p - T_{\text{max}}}{T_{\text{max}} - T_{\text{min}}}, & T_p > T_{\text{max}} \\ 0, & T_{\text{min}} \leq T_p \leq T_{\text{max}} \end{cases} \quad (16)$$

penalizes deviations from the desired window $[T_{\text{min}}, T_{\text{max}}]$ (e.g., $[20^\circ\text{C}, 50^\circ\text{C}]$).

The global slope between -40°C and 125°C ,

$$S = \frac{V_{\text{out}}(125^\circ\text{C}) - V_{\text{out}}(-40^\circ\text{C})}{165^\circ\text{C}}, \quad (17)$$

is used to determine if $V_{\text{out}}(T)$ is overall increasing or decreasing. Resistors R_1 and R_2 are then updated, scaled by J_{tp} , in directions that move T_p into the window while also flattening S . The third resistor R_3 is adjusted based on the deviation of $V_{\text{out}}(T_p)$ from V_{ref} to maintain the reference level. After each resistor update, an inner loop resmooths the curve by increasing $f_{n_m_b}$ until J_{smooth} returns below its threshold.

VI. NOISE AND 1/F BANDWIDTH OPTIMIZATION

A. Algorithm Overview

The `optimise_noise_with_optbandgap()` function implements nested-loop refinement starting from `optimize_bandgap()` geometry. It addresses three `spec.py` constraints: integrated RMS noise, spot noise specifications, and 1/f corner while preserving DC temperature characteristics.

Core loop:

- 1) `optimize_bandgap()` → DC baseline
- 2) Parse Spectre noise → spectra + contributions
- 3) Verify constraints → RMS, spot, flicker
- 4) Scale dominant devices → L(th), L+W(fn)
- 5) Repeat until convergence

B. Constraint Verification

a) Integrated RMS Noise:

$$\text{RMS}_{\text{noise}} = \sqrt{\int_{f_{\text{start}}}^{f_{\text{stop}}} S_{\text{total}}(f) df} \quad (18)$$

$\text{RMS} \leq \text{spec.integ_noise}$. Scale factor:

$$\text{scale}_{\text{RMS}} = \left(\frac{\text{spec.integ_noise}}{\text{RMS}}\right)^2 \times 0.3 \quad (19)$$

Algorithm 1 Three-Stage Bandgap Optimization

```

1: Initialize  $f_{n\_m\_b}, f_{p\_m}, R_1, R_2, R_3$  from pre-sizing.
2: repeat
  {Stage 1: smoothness}
3: Run Spectre temperature sweep with current parameters.
4: Extract  $V_{out}(T)$  and compute  $J_{smooth}$ .
5: if  $J_{smooth} > J_{smooth,max}$  then
6:   Increase  $f_{n\_m\_b}$  (mirror bias fingers).
7: end if
8: until  $J_{smooth} \leq J_{smooth,max}$  or iteration limit
9: repeat
  {Stage 2: reference centering}
10: Evaluate or interpolate  $V_{out}(30^\circ\text{C})$ .
11: if  $V_{out}(30^\circ\text{C}) < V_{ref,min}$  or  $> V_{ref,max}$  then
12:   Adjust  $f_{p\_m}$  (PMOS mirror fingers).
13: end if
14: Re-run Stage 1 if smoothness violated.
15: until  $V_{out}(30^\circ\text{C})$  within bounds or iteration limit
16: repeat
  {Stage 3: turning-point control}
17: Compute turning-point temperature  $T_p$  from slope sign changes.
18: Compute  $J_{tp}(T_p)$  and global slope  $S$ .
19: Update  $R_1, R_2$  based on  $S$  and  $J_{tp}$  to move  $T_p$  into window.
20: Update  $R_3$  to correct  $V_{out}(T_p)$  toward  $V_{ref}$ .
21: Re-run Stage 1 to restore smoothness.
22: until  $T_p$  within window or iteration limit

```

b) *Spot Noise*: For $i = 0 \dots \text{spec.no_points} - 1$:

$$S_{total}(\text{spec.entry}[i][0]) \leq \text{spec.entry}[i][1] \quad (20)$$

Scale: $\text{scale}_n = \frac{S_{actual} - S_{spec}}{S_{actual}} \times 0.3$.

c) *Flicker Corner*:

$$f_c = \arg \min_f |S_{fn}(f) - S_{th}(f)| \leq \text{spec.noise_corner} \quad (21)$$

C. Device Targeting and Scaling

Targets 7 high-contributors (Table I):

TABLE I
NOISE OPTIMIZATION TARGETS

Instance	Block	Scaling
I9.M2, I9.M0	PMOS Mirror	L(th), L+W(fn)
I5.M7, I5.M5	NMOS OpAmp	L(th), L+W(fn)
I5.M0	PMOS Load	L(th), L+W(fn)
I5.M14	PMOS Input	L(th), L+W(fn)
I9.M6	Cascode	L(th), L+W(fn)

Scaling Rule:

- Thermal: $L \times (1 + c\% \times \text{scale})$
- Flicker: $L, W \times (1 + c\% \times \text{scale})$

where $c\%$ = contribution percentage.

TABLE II
DC PERFORMANCE SUMMARY AFTER OPTIMIZATION

Parameter	Value
Supply voltage V_{DD}	1.2 V
Reference voltage V_{ref}	≈ 0.5 V (within spec)
Temperature range	-40°C to 125°C
Peak-to-peak V_{out} span	$\leq \text{spec. limit (via } v_{out_within_5mV})$
Turning-point temperature T_p	within $[20^\circ\text{C}, 50^\circ\text{C}]$
Power consumption	$\leq \text{spec.Max_power}$

D. Convergence Criteria

ALL must hold:

- $\text{RMS} \leq \text{spec.integ_noise}$
- $\forall i: \text{spot}[i] \leq \text{spec.entry}[i][1]$
- $f_c \leq \text{spec.noise_corner}$
- DC span: $\max V_{out}(T) - \min V_{out}(T) \leq 5$ mV
- $P \leq \text{spec.Max_power}$

Control: 50 max iterations, $\alpha = 0.3$ damping.

E. Performance

Typical convergence: 10-25 iterations (20-50 Spectre runs). Each iteration preserves DC via nested `optimize_bandgap()` call.

Returns complete geometry:

```

{"params": {L_p_o, L_p_m, W_p_o, ...},
 "noise_rms", "flick_bw", "dict_t_vs_vout"}

```

Advantages: Knowledge-driven, spec.py-flexible, DC-preserving, scalable.

VII. RESULTS

A. Simulation Setup

The proposed flow was implemented in Python and interfaced with Cadence Spectre running a CMOS current-mode bandgap reference designed for a 1.2V supply. The analytical pre-sizing script (`CALCULATIONS.py`) generated initial values for R_1, R_2, R_3 , branch currents, and device lengths, which were then passed to the gm/Id-based synthesis and optimization script (`AUTOBAND.py`). DC temperature sweeps were performed from -40°C to 125°C , and noise analyses were run over the required frequency band using Spectre's noise analysis.

B. DC and Temperature Performance

Fig. 4 shows the bandgap output voltage as a function of temperature before and after the automated optimization. The initial hand-sized design exhibits significant curvature and a large variation of V_{out} over temperature. After the three-stage optimization, the curve is smoothed, the turning point is placed inside the $[20^\circ\text{C}, 50^\circ\text{C}]$ window, and the overall temperature coefficient is reduced.

Table II summarizes the key DC performance metrics obtained from the automated flow.

The span of V_{out} across temperature is directly computed by the script using the function `vout_within_5mV()`, which evaluates the maximum minus minimum of the simulated

TABLE III
NOISE AND 1/F CORNER AFTER OPTIMIZATION

Parameter	Value
Total output spot noise at target f	$\leq \text{spec.Max_noise}$
integrated noise	$\leq \text{spec.integ_noise}$
Flicker corner / 1/f corner	$\leq \text{spec.noise_corner}$
Noise contributors	dominated by resized MOS devices

TABLE IV
RESULTS FOR $V_{DD} = 1.2\text{ V}$

Parameter	Expected	Output
Reference Voltage (27°C)	0.5V	0.499V
Peak-to-Peak V_{out} Span	5mV	3 mV
Power Consumption	500 μW	292.8 μW
Noise Corner	10KHz	9.20 KHz
RMS Noise	50 μV	42.99

$V_{out}(T)$ values. The final design satisfies the specified bound on this span while staying within the allowed power budget.

C. Noise Spectrum and 1/f Corner

The optimization framework also evaluates noise performance by parsing the PSF noise file. The function `parse_psf_noise_ascii()` reconstructs the total, flicker, and thermal noise density at the output node across frequency, and `get_out_total_noise_at_freq()` interpolates the total noise at a target low frequency. The 1/f noise bandwidth is extracted using `find_flicker_bandwidth()`, which searches for the crossing point between flicker and thermal noise.

Fig. 7 illustrates the final output noise spectrum, including total, flicker, and thermal components. Table III reports the main noise-related metrics after optimization.

The iterative `optimize_noise_with_optbandgap()` loop successfully reduces both the total noise and the flicker corner by selectively increasing the lengths and widths of the devices identified as dominant noise contributors, while maintaining the DC and temperature characteristics enforced by `optimize_bandgap()`.

D. Convergence and Automation Benefits

The complete flow—from analytical pre-sizing through gm/Id-based synthesis, three-stage DC optimization, and noise refinement—runs without manual intervention once the specification file (`spec.py`) is defined. The number of Spectre runs is bounded by user-configurable iteration limits in `optimize_bandgap()` and `optimize_noise_with_optbandgap()`, and in practice converges in a modest number of iterations, making the approach practical for iterative design and what-if exploration.

VIII. CONCLUSION

An automated design flow for a current-mode bandgap reference has been presented, combining analytical bandgap theory, gm/Id-based device synthesis, and a multi-stage simulation-driven optimizer. The approach encodes standard analog

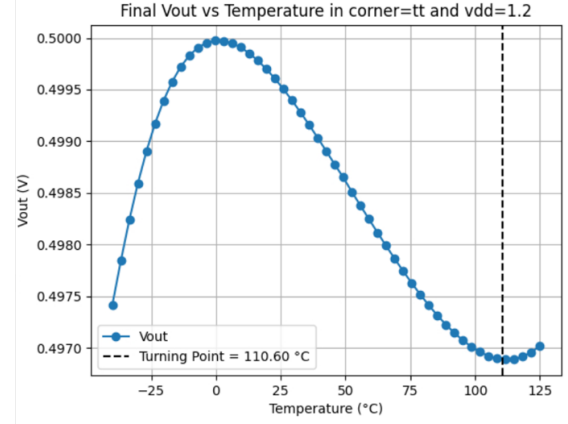


Fig. 4. Simulated bandgap output voltage versus temperature after optimization ($V_{DD} = 1.2\text{ V}$).

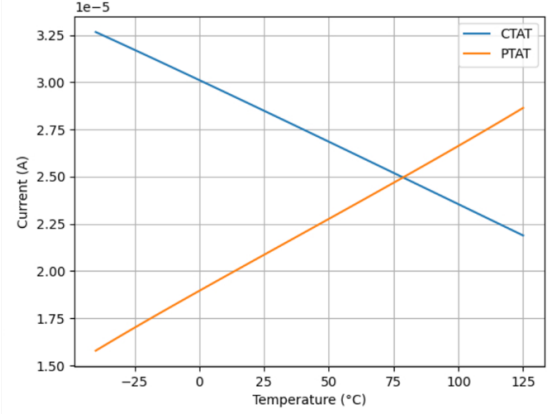


Fig. 5. CTAT vs PTAT ($V_{DD} = 1.2\text{ V}$).

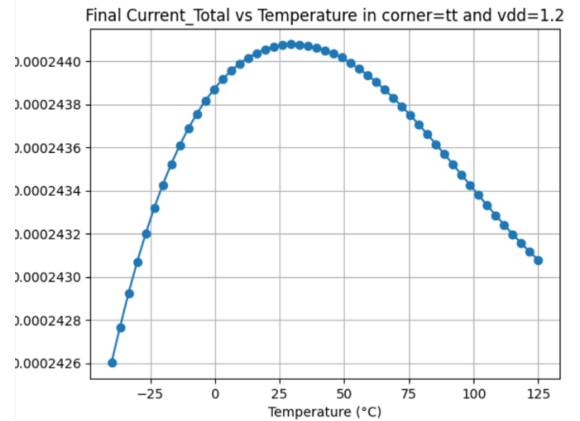


Fig. 6. Total current consumed.

design heuristics into a Python–Spectre loop, achieving a temperature-stable, low-noise reference with minimal manual tuning.

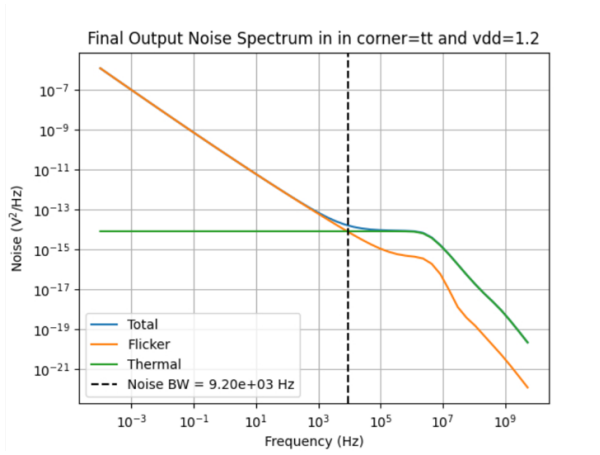


Fig. 7. Final output noise spectral density showing total, flicker, and thermal components.

IX. FUTURE WORKS

A startup circuit can be added, and the framework can be extended to incorporate PVT corners and mismatch analysis, post-layout parasitics, and more advanced search strategies such as surrogate modelling and global optimization, while preserving the effective structure of analytical pre-sizing and closed-loop refinement.